

Resource Contracts for Completing Optimistic Computations

ANONYMOUS AUTHOR(S)

An optimistic computation can retry, recompute under protection, or validate a prepared result under protection. These established mechanisms leave a resource-contract question: what must one policy spend to guarantee completion and optimal protected work at every unknown interference budget? For persistent whole-kernel jobs, we derive the exact least total-work curve for independent jobs, with a lower bound covering early preparation and retained records. On every dependency DAG, we characterize all safe completing modes for an explicit residual contract: incurred protection must not exceed the top- k completed work, where k counts observed cached mismatches. The resulting filter needs no future job weights or edges and supports constant-time mode queries. A common mismatch fee preserves this interface; unrestricted fees can make identical observed histories require opposite choices because of future costs. Conditional Java correspondence and native Linux/Roslyn studies identify the applicable call, work and observation contracts. More permissive choices do not by themselves improve worst total work, and the native studies establish neither general cost transfer nor latency gains.

CCS Concepts: • **Software and its engineering** → **Concurrent programming structures**.

Additional Key Words and Phrases: concurrent programming, retries, resource guarantees, scheduling, online policies

1 Introduction

A library that completes optimistic computations must decide when to retry, when to compute under protection, and when to validate a prepared result before protected recomputation. These mechanisms already exist in locking and irrevocable transactions [33, 44]. We ask what contract a policy can provide using them: a fixed allowance of unsuccessful calls, a bound on protected kernel work at every unknown number of interfering writes, and the total work that these promises require.

These are different resources. One unit-work job illustrates the choice. Retrying once and then computing under protection uses at most two bodies but protects a body after one interfering write. Preparing again before a guarded cached completion instead protects no body after one write, yet can execute three bodies after two writes. Both have two logical completion calls. A retry counter alone therefore does not determine the work contract.

The same distinction appears in Linux’s single rename-lock pathname loop [7, 8]. Its upstream fallback allows at most two body attempts. Our cached variant permits a third attempt to reduce protected rebuilding after one write. This is a relaxation of a source-derived bound, not an unchanged-contract speedup. Roslyn has a related immutable-solution retry loop, but lazy work and guarded merges prevent exact fixed-work transfer; dense-arrival measurements favor the simpler two-mode variant. These cases motivate examining which contracts an interface actually supports, rather than treating a logical call as a duration or a native attempt.

We study finite dependent jobs with persistent saved outputs and positive whole-kernel work. The writer’s total interference budget is hidden. A caller supplies retry slack r , allowing at most $n + r$ completion calls for n jobs. After r failures, completing cached operations attain the optimal protected-work curve: the sum of the largest $(B - r)_+$ job weights under at most B writes. We ask what follows when *one policy must keep that optimal curve at every budget*, and how a scheduler can preserve it while selecting other completing modes.

The results answer three connected questions.

- (1) *What total work does the contract force?* Independent jobs admit an exact least worst-work curve, attained simultaneously at all budgets by one sorted policy. The lower bound

covers every program in the declared interface, including retained records and paid early preparation.

- (2) *What information must a safety component receive?* On every DAG, the exact residual safe-mode set depends only on completed weights, incurred protection, observed mismatches and the proposed job. No future weight or edge is needed. Queries take $O(1)$ operations and completion updates take $O(\log n)$.
- (3) *When does that interface stop being exact?* A common additive mismatch fee cancels, and a wider fee-separation condition preserves the local rule. With unrestricted fees, two reachable identical prefixes require opposite present choices even with the same full mismatch-cost vector and DAG. Future fee information then matters.

For independent works 8 and 16 with no spare calls, one universally optimal-protection policy requires worst $(W, L) = (40, 16)$ at $B = 1$. If $B = 1$ is supplied, the policy in Figure 1 attains $(32, 16)$. The difference measures the price of the simultaneous protection contract, not the price of merely omitting a budget argument.

Top-budget sums, adaptive testing and safety-game filters are inherited. Our contribution is their exact operational characterization for this completion contract, including what can be hidden from its enforcing component. Conditional Java wrappers make the contract executable; they do not introduce a new synchronization primitive. An actual client requiring this entire combination of guarantees remains unestablished. Section 10 makes the closest-work comparisons explicit. The earlier development snapshot [22] studies premium-weighted work with retained guards; the present results use per-call release and separate resources. The accompanying archive preserves the earlier frontier, domination and charged-complexity results [23].

2 Completion Interfaces and Resource Contracts

Write $\mathbb{N} = \{0, 1, \dots\}$. Jobs $J = \{1, \dots, n\}$ form a finite DAG, $n \geq 1$. Job i is ready when every predecessor in $\text{Pred}(i)$ has completed. Each completion saves an immutable output despite later writes. Completion requires one transformation per job, without a simultaneous snapshot. Policies are deterministic and causal.

Each ready job has a pure, opaque whole kernel on its own input and exact saved predecessor outputs. Every execution performs work $w_i > 0$, inside or outside protection. Partial/incremental evaluation, preparation before readiness and external completion are excluded. A prepared result certifies those inputs; subsequent map reads cannot reconstruct saved outputs.

State and transition notation. Write $\sigma = (x, \mathcal{M}, \mathcal{R}, \lambda)$ for live own-input identities, completed outputs, a finite snapshot/record store and program locals. A record $\rho = (i, s, \mathbf{u}_i, z)$ certifies a completed kernel execution on captured identity s and saved predecessor tuple $\mathbf{u}_i = \mathcal{M}|_{\text{Pred}(i)}$, returning z . Copies retain this certificate; fabrication or retagging requires new computation. Define $\text{pub}_i(z)$ to assign $x_i := z$, save $\mathcal{M}(i) := z$ and complete i . Only publishing completion changes an unfinished live key through a foreground action. Table 1 gives effects, observations and charges; $m = [s = x_i]$ is exact identity comparison. A failed cheap call clears selection, while its retained record copies remain available.

A turn and its write gate. The program selects an action a , fixing its job and mode. The environment then chooses a finite legal write word ω before that action's effect:

$$\sigma \xrightarrow{\omega} \widehat{\sigma} \xrightarrow{a/o, \delta} \sigma'.$$

Here o is the returned observation and $\delta = \Delta(Q, W, L)$. Each external assignment changes its own x_i and leaves $\mathcal{M}$ immutable. At every finite history an unfinished target admits an identity fresh

to all its prior identities, records and raw snapshots, including snapshots predating the charged interval. The adversary may choose never to reuse one; finiteness alone does not provide this supply.

After the gate, record selection uses only the already-computed prior store. A new unprotected preparation, even if paid, is a separate action with a later comparison gate. Completing comparison/computation/publication is indivisible to writes; every guard ends on return. Inspections honestly return live entries. Cached completion reveals its output and comparison Boolean; retained values, reference equality and local state remain available. Clocks, allocation identifiers and work counters are not observations. Section 7.1 separately restricts cached observations.

Preparation and charging. General programs may prepare early after readiness, using immutable own snapshots and saved parents; raw snapshots may be old. Immediate preparation captures and computes only for the next call. A pure unprotected body has no live interaction: at completed-operation boundaries, contract it with its returned record and charge, and replay intervening writes before the next action. Section 8 accounts for pending writes without placing a completed result at an unfinished source prefix.

The common start precedes all computed initialization. W charges every kernel, including discarded/early computations; L charges its protected part. Q counts validation/publication/completion calls, including failures, but not preparations or writers. Waiting, common call overhead and nonkernel allocation are outside these coordinates. Write $\Omega_k(T)$ for the sum of the largest $\min(k, |T|)$ works, $\Omega(T)$ for total work, and $\Omega_0 = 0$; omitted sets mean J . These aggregate objects are inherited [13, 14].

Table 1. Transitions after the write gate. A selected record is $\rho = (i, s, \mathbf{u}_i, z)$; $m = [s = x_i]$ and $v_i = (1 - m)w_i$. Kernel/completion actions require a ready unfinished job; inspection may read any key. z' is a new kernel result.

Action	Effect	Observation	$\Delta(Q, W, L)$
Inspect i	No state change	x_i	$(0, 0, 0)$
Prepare i	Run kernel on captured $(s, \mathbf{u}_i)$; add ρ to $\mathcal{R}$	ρ	$(0, w_i, 0)$
Cheap i, ρ	If m , $\text{pub}_i(z)$; otherwise clear selection	m	$(1, 0, 0)$
Fresh i	Run kernel on $(x_i, \mathbf{u}_i)$; $\text{pub}_i(z')$	z'	$(1, w_i, w_i)$
Cached i, ρ	If m , $\text{pub}_i(z)$; else run kernel and $\text{pub}_i(z')$	Output and m	$(1, v_i, v_i)$

Free copying, selection and local actions modify only $\mathcal{R}, \lambda$. Two modes allow cheap/fresh; three add cached. “Cheap” refers to kernel work. Direct locking, retained guards, cross-job callbacks and outside completion are excluded. Selected operations terminate. Admissibility requires completion of every job after finitely many actions on each admitted play; infinite free actions fail it. No uniform action bound over all environments is assumed.

Strategies and admissibility. B bounds external writes; r permits extra failed calls beyond n completions. An informed strategy is $P(B, h)$; an unaware one is $P(h)$. Let $\mathcal{E}_b$ contain gate strategies using at most b writes on every play. For mode family j , $\mathcal{P}_j(B, r)$ requires completion with $Q \leq n + r$ against every $E \in \mathcal{E}_b$. In contrast, one $P \in \mathcal{U}_j(r)$ must satisfy that cap for every $b \geq 0$ and $E \in \mathcal{E}_b$, without receiving B . The gate adversary knows the deterministic program and its prior store. A fixed adverse play can be scripted at gates; no wall-clock-oblivious or randomized value follows. Its finite write count also bounds a capped environment preserving that play. Worst protection is $\sup_{E \in \mathcal{E}_B} L(P, E)$.

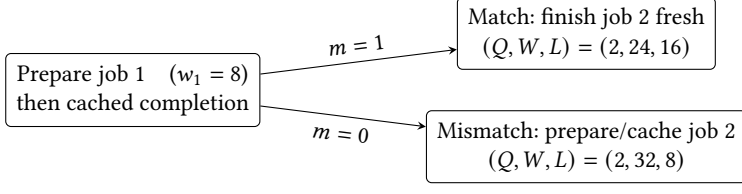


Fig. 1. One policy for independent works (8, 16), supplied $B = 1$, $r = 0$. A mismatch certifies that the one allowed write occurred between preparation and comparison. The remaining cached call then matches. The two paths give separate worst bounds $(\sup W, \sup L) = (32, 16)$; they do not have to attain both on the same execution.

LEMMA 1 (FRESH-WRITE AND FINITE-ADVERSARY FACTS). *In Section 2's game, a fresh own-key write at a selected i -operation's gate invalidates every retained i -record. Any concrete prefix has both a no-write and a fresh-write continuation whenever that extra write respects the environment's budget. An adversary bounded by b writes on every play forces any $P \in \mathcal{P}_j(b, r)$ to finish every job, regardless of free inspections or record retention.*

PROOF. A fresh identity differs from every retained target identity. Copying/selection cannot change that, and all observations and job/mode choices agree until the gate. The bounded adversary belongs to $\mathcal{E}_b$; leaving any job unfinished violates admissibility even with infinitely many free actions. Thus every job completes through Table 1. No unbounded adversary is truncated. $\square$

LEMMA 2 (WORK CHARGED BY USEFUL COMPARISON WRITES). *From the charged common start, consider a complete execution in which the adversary writes only fresh own-key identities at comparisons whose zero-write continuation would match. Let H be the multiset of resulting cheap failures and D the set of resulting cached mismatches. Then*

$$W \geq \Omega + \sum_{h \in H} w_{i(h)} + \sum_{i \in D} w_i.$$

Already-stale failures may consume calls, but need no extra charge in this inequality.

PROOF. At each useful comparison, charge the record that the no-write outcome of *that same call* would select, even if its failure branch selects an older one. The creating calculation is in the pre-gate store and is therefore paid on both branches. The fresh write invalidates every prior target record. Before completion, neither failed cheap calls nor other foreground actions restore an earlier live identity; the restricted adversary never restores one. Index a charged preparation by its job, captured identity and creating invocation. Later useful comparisons at that job have different identities, so cannot charge the same invocation or a copy of it. Final completion needs a further distinct execution: protected work or a matching preparation for the final identity. A cached mismatch charges its preparation as duplicate work and its recomputation within mandatory Ω . Different jobs cannot share charges. These calculation instances are pairwise distinct across useful failures, cached mismatches and mandatory completions. A stale cached call supplies its mandatory protected calculation; its old preparation is unused slack in the bound. This handles old snapshots and early computation globally; a suffix may have prepaid records. $\square$

3 Universal Call and Protected-Work Guarantees

For unfinished jobs S , let $\mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$ denote the ready jobs.

THEOREM 1 (PROTECTED WORK UNDER A CALL CAP). *In Section 2's interface, fix integers $B, r \geq 0$. Among programs supplied with B that complete all jobs with $Q \leq n + r$ in every environment with at most B writes, the minimum worst protected work is*

$$L_3^*(B, r) = \Omega_{\min((B-r)_+, n)}, \quad L_2^*(B, r) = \begin{cases} 0, & B \leq r, \\ \Omega, & B > r, \end{cases}$$

with and without cached completion, respectively. Any DAG is permitted.

PROOF. At most r calls may fail to complete. Freshly prepared cheap attempts have disjoint read-to-comparison intervals; each failure needs a distinct write. If $B \leq r$ they give $Q \leq n + B$, $L = 0$. Otherwise switch after r failures to cached completion: at most $B - r$ writes remain, giving the stated bound. All-protected gives the two-mode upper bound.

For two modes when $B > r$, reject cheap calls with fresh writes, stopping unconditionally after $r + 1 \leq B$ rejections. The call cap forbids exhausting this budget, so every job completes under protection.

For three modes, target the $k = \min(B - r, n) > 0$ heaviest jobs. Count noncompleting target comparisons f and completed targets p , initially zero. Write a fresh identity only at target comparison gates while $f \leq r$ and $p < k$; stop permanently at $f = r + 1$ or $p = k$. Counters follow actual outcomes, including fresh completions. While active, cheap target calls fail, cached calls recompute, and fresh completion protects its body without a write.

At operation boundaries the write count satisfies $v \leq f + p$: each write caused a failure or cached completion. Before another write, $v + 1 \leq f + p + 1 \leq r + k \leq B$. This is unconditionally an $\mathcal{E}_B$ strategy, even on plays violating the call cap.

Admissibility excludes $f = r + 1$: those failures plus n mandatory completions violate the cap. Lemma 1 forces every target to complete, fresh or by cached mismatch, protecting its body. Hence $p = k$ and $L \geq \Omega_k$. Free inspections, stale selections and retained copies cannot evade this. Predecessors contribute their own mandatory calls; target predecessors are counted once in p . $\square$

THEOREM 2 (ONE POLICY FOR EVERY WRITER BUDGET). *Let $\mathcal{U}_j(r)$ contain the j -mode programs that receive no B and complete with $Q \leq n + r$ in every finite-write environment. Set $U_j(B, r) = \inf_{P \in \mathcal{U}_j(r)} \sup_{\# \text{writes} \leq B} L(P, E)$. Then*

$$U_3(B, r) = L_3^*(B, r), \quad U_2(B, r) = \Omega \mathbf{1}[B \geq r].$$

One policy per mode family attains these values simultaneously for all B and positive works on any DAG, using only readiness and a global cheap-failure counter.

PROOF. Switch after r cheap failures to a fresh read/preparation with cached completion (three) or to fresh protected completion (two). This uses at most r extra calls universally. At $B < r$ no protection is needed; at $B = r$ cached completions after switching still match. Theorem 1 supplies the remaining upper and three-mode lower bounds.

For two modes when $B \geq r$, reject every cheap call with a fresh own-key write until r rejections, then stop. Only protected calls complete before this. Afterwards, another rejectable call could fail in a different finite environment, violating the call cap. Thus all jobs complete under protection with at most $r \leq B$ writes. That additional hypothetical write is absent from the lower-bound execution. Retained preparations and inspections cannot prevent a fresh write before comparison. $\square$

For equal works and $B > r$, the protection ratio is $n/\min(B - r, n)$. At $B = r$, informed two-mode policies know the budget is exhausted; unaware ones must tolerate another write. Roslyn reports distinct per-update/shared-batch caps.

4 The Total Work Needed for Optimal Protection

We minimize total work under simultaneous or supplied-budget protection contracts, for arbitrary r . As Figure 1 shows, separate suprema may have different witnesses; they do not determine $\sup(W + \lambda L)$.

Two bodies and two calls are different contracts. For one unit-work job, a B -unaware program capped at two body executions under every finite write budget must protect a body already at $B = 1$. Otherwise its first matching comparison is cheap: a gate write would force a cached body. Force that cheap failure, then stop writing. Finite unprotected completion requires another matching record. A second bounded environment invalidates that record too. Lemma 2 charges two distinct invalidated preparations and a mandatory final body, contradicting the two-body cap. The environments use at most one/two writes; no call cap is assumed.

Cheap followed by fresh attains the two-body cap. Repreparing for cached completion instead gives worst (W, L) pairs $(1, 0)$, $(2, 0)$ and $(3, 1)$ at budgets 0, 1 and $B \geq 2$, still with $Q \leq 2$. The whole-kernel premise excludes a native lower bound for partial snapshots or variable body work.

4.1 One Policy with Optimal Curves at Every Budget

Let $\mathcal{V}_r$ contain the policies in $\mathcal{U}_3(r)$ satisfying $L(P, E) \leq \Omega_{(b-r)_+}$ for every $b \geq 0$ and $E \in \mathcal{E}_b$. This requires one *simultaneous optimal-protection curve*; it is stronger than universal completion. Write $W_P(B) = \sup_{E \in \mathcal{E}_B} W(P, E)$.

LEMMA 3 (CAPPING A REALIZED PLAY). *For $P \in \mathcal{V}_r$, a finite prefix containing d actual writes has protected work at most $\Omega_{(d-r)_+}$.*

PROOF. Continue the prefix without writes. Admissibility forces completion. Cap that environment unconditionally at d : induction preserves each action and realized write, and there are no later writes to remove. The resulting environment belongs to $\mathcal{E}_d$ and preserves the complete play. Its total protected work satisfies the bound; nonnegative costs give the prefix bound. The program need not observe d . $\square$

THEOREM 3 (LEAST TOTAL-WORK CURVE UNDER SIMULTANEOUS OPTIMAL PROTECTION). *For independent jobs with sorted positive works $a_1 \leq \dots \leq a_n$,*

$$\inf_{P \in \mathcal{V}_r} W_P(B) = F_r(B) := \begin{cases} \Omega + Ba_n, & B \leq r, \\ \Omega + \max_{1 \leq m \leq \min(B-r, n)} \{ra_{n-m+1} + \Omega_m\}, & B > r. \end{cases}$$

One policy attains this entire curve: process jobs in ascending work order, freshly prepare before each cheap call until r cheap failures, and freshly prepare before each cached completion thereafter. It observes neither B nor cached comparison results. Sorting, prefix sums and prefix maxima construct the curve in $O(n \log n)$ arithmetic/comparison operations, representing its initial linear segment analytically.

PROOF. *Upper bound.* Each bad comparison needs a write in its distinct preparation-to-comparison interval. There are at most r cheap failures; cached mismatches occur only thereafter, at distinct jobs. Thus $Q \leq n + r$ and $L \leq \Omega_{(B-r)_+}$ at every budget. When $B \leq r$, at most B preparations are lost, each costing at most a_n .

For $B > r$, suppose cached mode starts at rank j and has $m \geq 1$ mismatches. All failed preparations were at ranks at most j and the m mismatched jobs lie in the suffix, so $j \leq n - m + 1$. Duplicate work is at most $ra_j + \Omega_m \leq ra_{n-m+1} + \Omega_m$, with $m \leq B - r$. Without a mismatch it is at most ra_n , below the $m = 1$ term. For each permitted m , match the first $n - m$ jobs, fail cheap r times at the next job, and mismatch all m cached completions. This attains its term using $r + m$ writes. The analogous maximum-job witness attains $\Omega + Ba_n$ at $B \leq r$.

Lower bound for arbitrary programs. Fix $m \leq \min(B - r, n)$ and target the m heaviest jobs. Write a fresh own-key identity only at target comparisons whose zero-write continuation would match. Let f count these useful cheap failures, c useful cached mismatches and t all target completions. Stop forever after $f = r + 1$ or $t = m$. At operation boundaries the write count is $d = f + c$, with $c \leq t$. Before another write, $d + 1 \leq f + t + 1 \leq r + m$. This is a globally $(r + m)$ -bounded adversary even on inadmissible programs.

An admissible program cannot reach $f = r + 1$, and every target eventually completes. Before all targets finish, no target comparison can match: a useful comparison is invalidated, and a stale one already fails or protects. Thus all targets complete protectively and $L \geq \Omega_m$. Lemma 3, applied at the realized final count d , gives

$$\Omega_m \leq L \leq \Omega_{(d-r)_+}, \quad d = f + c \leq r + m.$$

Positivity forces $d = r + m$, hence $f = r$ and $c = m$. All target completions are useful cached mismatches; this is a consequence of the contract, not an assumed normal form. Lemma 2 now charges r distinct failed preparations of work at least a_{n-m+1} , all m duplicate target kernels and mandatory Ω . Maximizing over m gives the lower bound.

Early budgets. At $d \leq r$, zero protection forbids fresh or mismatching cached completion. The $m = 1$ witness just constructed has r useful cheap failures at a maximum-work job, then its useful cached mismatch. Truncate after the first $B \leq r$ writes. Its unchanged prefix has destroyed B distinct paid preparations of that unfinished job. Eventual completion needs another distinct kernel, giving $\Omega + Ba_n$ by global charging. At $B = 0$ mandatory work suffices. Lower bounds include retained records, free inspections and paid early preparations after readiness. $\square$

The maximizing m need not be largest: $(1, 100)$ and $r = 1$ give terms 200 and 102. The least W/L curves may have different maximizing executions. Other optimal- L policies can use more work; relaxing protection changes the comparison.

Dependency boundary. The lower bound is order-independent. For any DAG, a topological cheap-then-cached policy attains $F_r(B) = \Omega + Bw_{\max}$ at $B \leq r + 1$, retaining optimal L at all budgets. At $r = 0$, all-cached gives $F_0(B) = \Omega + \Omega_B$. For positive retries, precedence can increase later values and make cached observations matter; Proposition 7 gives exact contrasting curves. A least entire curve for arbitrary positive-retry DAGs remains unresolved.

5 A Safety Filter That Needs No Future Job Data

The least independent-job curve answers the cost of a contract. On a DAG, a scheduler must also decide whether its next protected computation preserves that contract. Solving a new game whenever the ready set changes is unnecessary for the following residual interface.

Fix the finite instance J and a completed ideal D , with $S = J \setminus D$. Let k count cached mismatches since leaving the cheap phase and ℓ be protected work incurred. In this section, each remaining job completes once, either fresh or by *current capture and immediate preparation* followed by cached completion. No further cheap calls, early preparations, retained guards or partial kernels are allowed. A scheduler can use its available history to choose a ready job in finite time. It receives no future write bound.

The residual contract. A state is viable if there exists one such completing policy P satisfying

$$\forall b \geq 0 \forall E \in \mathcal{E}_b : \quad \ell + L_{\text{future}}(P, E) \leq \Omega_{k+b}(J). \quad (1)$$

Here $\mathcal{E}_b$ concerns future writes and has an unconditional cap on every play. The same P must work for every b . This is an explicit interface obligation: it does not assert that k is the total number

of past writes. A history exposing additional writes can support a weaker global obligation, so necessity below is local to (1).

LEMMA 4 (A SORTED-UNION IDENTITY). For $0 \leq k \leq |D|$,

$$\min_{0 \leq m \leq |S|} \{\Omega_{k+m}(J) - \Omega_m(S)\} = \Omega_k(D).$$

PROOF. The largest k elements of D and m of S form a $(k+m)$ -element subset, proving the lower bound. If $k = 0$, use $m = 0$. Otherwise let τ be the k th largest completed weight and take the m unfinished weights strictly greater than τ . These, together with the k largest completed weights, are $k+m$ largest weights of J , with ties assigned to D . This attains equality, and $k+m \leq n$. $\square$

THEOREM 4 (EXACT RESIDUAL SAFETY). On every DAG, for $0 \leq k \leq |D|$, the following are equivalent:

- (1) some residual policy satisfies (1);
- (2) all-cached completion in any causal ready order satisfies it;
- (3) $\ell \leq \Omega_k(D)$.

From a viable state every ready cached action is safe. A ready fresh action i has a safe continuation exactly when

$$\ell + w_i \leq \Omega_k(D \cup \{i\}). \quad (2)$$

Thus this predicate, together with all cached actions, is maximally permissive for the stated residual contract and action class.

PROOF. Under all-cached, b writes can mismatch at most b distinct jobs, costing at most $\Omega_b(S)$. Targeting the heaviest m remaining jobs as they become ready attains $\Omega_m(S)$ with m writes. Lemma 4 therefore gives the exact all-cached condition $\ell \leq \Omega_k(D)$.

For necessity against any residual policy, fix m and the m heaviest jobs of S . The environment has one write slot per target. If that target selects cached, spend its slot on a fresh own-key identity after current preparation; if it selects fresh, spend nothing. This environment uses at most m writes on every play. Finite completion forces every target to finish and protect its body in either mode. Consequently (1) requires $\ell + \Omega_m(S) \leq \Omega_{k+m}(J)$ for every m . Lemma 4 gives the condition.

A cached match enlarges D with unchanged k, ℓ . A mismatch adds i to D , adds w_i to ℓ and increments k ; selecting i together with the k largest old jobs gives $\Omega_{k+1}(D \cup \{i\}) \geq \Omega_k(D) + w_i$. Both outcomes remain viable. A fresh successor has $(D \cup \{i\}, k, \ell + w_i)$, so its viability is exactly (2). Each action completes one job; choosing only these actions terminates after $|S|$ calls and preserves the contract. $\square$

Composing the filter with retries. Use any immediate-preparation cheap controller until all jobs finish or r cheap failures occur. At the switch initialize $k = \ell = 0$ and retain all cheaply completed weights in D . The filter is initially viable. Every earlier cheap failure and later cached mismatch needs a different write: their current-capture/comparison intervals are disjoint. Hence $r + k \leq B$ whenever the switch occurs in a B -write execution. The invariant yields

$$Q \leq n + r, \quad L = \ell \leq \Omega_k(D) \leq \Omega_{(B-r)_+}(J).$$

If completion precedes the switch, $L = 0$. Theorem 1 shows that this protection curve is optimal even against the larger general-program class.

What the filter stores. Maintain the completed weights in two heaps: the largest k in a min-heap, the rest in a max-heap, together with the top- k sum T . For $k = 0$, positive fresh work is forbidden. Otherwise let τ be the minimum of the first heap; the fresh test is

$$\ell + w_i \leq T + \max\{0, w_i - \tau\}.$$

It takes $O(1)$ arithmetic/comparison operations. Completion inserts one weight, and a mismatch increases k by one; a constant number of heap transfers gives $O(\log(|D| + 1))$ update time and $O(|D|)$ storage. Starting empty needs no preprocessing. Integer bit costs are additional.

No future weight or edge enters this predicate. The scheduler checks readiness and proposes a job/mode; the safety component returns the permitted modes and updates only after completion. Thus the same component works when future jobs are undisclosed, provided each realized instance is finite and obeys the interface. This supplies an invariant for each finite instance; infinitely arriving jobs have no completion guarantee.

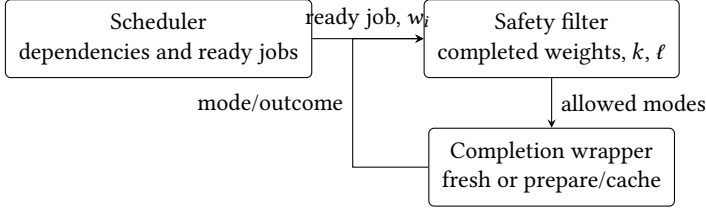


Fig. 2. The residual interface separates readiness from resource safety. Future weights and edges remain outside the filter. Every selected mode completes one job; the returned cached comparison updates k . The guarantee concerns the explicit residual contract, while the scheduler may optimize total work inside the permitted set.

Permissiveness and total work. The distinction is observable even on a chain. Let $r = 1$: cheaply complete weight 100, fail once at weight 1, then mismatch its cached completion. Now $k = \ell = 1$; a fresh weight 2 is safe because $1 + 2 \leq 100$. A rule that waits until *all* remaining work fits the current allowance would reject it when the remaining weights are $(2, 80, 80, 80)$. The smaller interface permits this decision without knowing those later weights.

Safe choices need not minimize W . However, any immediate fresh/cached suffix has worst total work no greater than all-cached after the *same cheap controller*. For a concrete execution retain its exact prefix with b_0 actual writes and duplicate cheap work C . If m suffix jobs mismatch, its work is at most $\Omega + C + \Omega_m(S)$. An all-cached continuation can attain this upper expression by targeting the m heaviest remaining jobs, using $b_0 + m \leq B$ writes. Script that prefix and cap the resulting writer unconditionally at $b_0 + m$. This proves worst-curve domination, not dominance under the same write timing. Therefore ascending cheap order retains Theorem 3's least curve for independent jobs under *any* filtered suffix; at $r = 0$ the same holds on every DAG. Positive-retry DAG work optimality remains open.

6 When Accounting Changes the Information Required

The filter's exactness depends on its cost contract. Keep the residual actions and write semantics, but charge fresh completion $p_i > 0$, cached match 0, and cached mismatch $q_i = p_i + h_i$ with $h_i \geq 0$. Let $T_j(A)$ sum the largest j values of q in A , and set $G_j = T_j(J)$. The benchmark is now the *all-cached charged curve*, not an asserted optimal charged curve. One policy must satisfy $\ell + L_{\text{future}} \leq G_{k+b}$ for all future b .

THEOREM 5 (A FEE CONDITION PRESERVING THE LOCAL INTERFACE). *For arbitrary nonnegative fees, $\ell \leq T_k(D)$ is sufficient, and the filter allowing every cached action and fresh i when*

$$\ell + p_i \leq T_k(D \cup \{i\}) \quad (3)$$

preserves the benchmark. If $0 \leq k \leq |D|$ and

$$\max_{i \in J} h_i \leq \min_{i \in J} q_i, \quad (4)$$

the condition is also necessary, so the filter is maximally permissive. Every common additive fee $h_i = h$ satisfies (4).

PROOF. The sufficient invariant follows from the same top-sum inequalities as Theorem 4, with mismatch increment q_i and fresh increment p_i . All-cached is safe exactly on this invariant, by Lemma 4 applied to q .

For necessity, suppose $\ell = T_k(D) + \delta$ with $\delta > 0$. Choose the equality cut m from Lemma 4, so $G_{k+m} = T_k(D) + T_m(S)$ and $k + m \leq n$. Fix any candidate policy and target the m largest remaining q_i . Write once at a target if its selected mode is cached and nowhere else. This is an unconditionally m -bounded environment, so the policy completes. Let c be its realized cached targets and $t = m - c$ its fresh targets. Target costs give

$$\ell + L_{\text{future}} \geq G_{k+m} + \delta - \sum_{\substack{i \text{ a fresh} \\ \text{target}}} h_i.$$

Cap this environment at its realized count c on every play. Induction preserves this completed play: every realized write was among the first c , and after the last one no write is removed. The policy receives no b , so its choices remain the same. Its contract therefore requires cost at most G_{k+c} .

Yet $G_{k+m} - G_{k+c}$ is the sum of t actual q values. Condition (4) makes it at least the sum of the t saved fees above. The preserved play exceeds G_{k+c} by at least δ , a contradiction. Equality in (4) is harmless. For common h , every $q_i = p_i + h > h$. $\square$

The admission contract and common fees. A library can declare one H and admit only jobs satisfying $h_i \leq H \leq q_i$. If completed and future jobs all meet this contract, the theorem applies without revealing their future exact costs. The two-heap implementation substitutes q_i for stored weights and p_i for the fresh increment. For a common h , actual cost is $\ell = L_{\text{body}} + hk$, while $T_k(D) = \Omega_k(D) + hk$. The fee cancels from both the invariant and fresh test: the body-only filter remains exact even for a large common fee. Merely saying “fees are nonzero” would miss this stable case.

PROPOSITION 6 (FUTURE FEES CAN CHANGE A PRESENT PERMISSION). *Over the unrestricted fee class, a filter reading only the visible prefix and completed/current costs cannot always return the exact safe-mode set. This remains true if it also knows the entire DAG and the entire vector q_J .*

PROOF. Use the chain $0 \rightarrow 1 \rightarrow 2$, no cheap calls, and $q = (2, 2, 4)$. Jobs 0 and 1 have $p = 1, h = 1$ in both worlds. In A, job 2 has $(p_2, h_2) = (1, 3)$; in B it has $(3, 1)$. Both benchmark curves are $(G_0, G_1, G_2, G_3) = (0, 4, 6, 8)$.

The shared prefix is cached 0 mismatching after one write. Thus $D = \{0\}$, $k = 1$, $\ell = 2$, and the current job 1 has identical costs. In A, fresh 1 followed by fresh 2 costs 4 and satisfies every future budget. A fully safe initial policy exists: cache 0; on mismatch finish 1 and 2 fresh; otherwise cache 1; on mismatch finish 2 fresh; otherwise cache 2. Its four leaves have costs 0, 4, 3, 4, with minimum write counts 0, 1, 1, 1. Every leaf completes in three calls. Harmless writes only increase the benchmark.

In B, fresh 1 leaves $\ell = 3$. Fresh 2 then costs 6, exceeding $G_1 = 4$ without another write. Cached 2 can be mismatched by one further write, costing 7 and exceeding $G_2 = 6$. Neither possible completing continuation is safe. The common prefix is nevertheless reachable under B’s all-cached safe policy. Identical filter inputs thus require opposite permissions; the hidden future fee split is decisive. $\square$

The sufficient filter rejects fresh 1 in both worlds. Its conservatism in A is real: it refuses leaving its invariant even though a later low-cost fresh completion can recover the contract. In particular, all-cached is not necessarily safe from every viable state outside that invariant. Condition (4) is sufficient for exact locality; we do not claim it is necessary for each individual instance.

What may be subtracted from a native cost. If each job pays one fixed fee g_i in every mode and outcome, the full benchmark must include $\sum_{i \in J} g_i$. Subtracting $\sum_{i \in D} g_i$ from incurred cost then gives the variable-cost game above. Cached-only validation, outcome-dependent restart costs, per-retry fees and variable waiting do not satisfy this baseline condition. Treating all of them as a free common constant would change the guarantee. This section supplies a priced interface theorem, not a fitted model of native guard duration.

7 Observation and Call Accounting Boundaries

7.1 What the Cached Comparison Reveals

Consider an erased observation contract. Every kernel invocation returns the same immutable per-job object z_i , which completion publishes over its live entry. The cached primitive's *entire visible transition* exposes only that output and completion: no compared input, Boolean, comparison-dependent record selection or callback-local side effect survives. Kernel internals, work counters, allocation IDs and clocks are hidden. Free outside inspections and old snapshots remain allowed. Constant output alone is insufficient if the callback saves the current input elsewhere.

PROPOSITION 7 (A STRICT OBSERVATION BOUNDARY). *For independent works (8, 16), supplied $B = 1$ and $Q \leq 2$, the visible-comparison frontier is $\{(24, 24), (32, 16)\}$; erasure changes it to $\{(24, 24), (40, 16)\}$.*

For $A \rightarrow U \rightarrow V$ and independent Z with $w_A = 5$, $w_U = w_V = 1$, $w_Z = 2$, one B-unaware policy with $Q \leq 5$ and optimal protection at every budget has least work curves

$$W_{\text{visible}} = (9, 14, 19, 20, 20), \quad W_{\text{erased}} = (9, 14, 19, 20, 21)$$

at $B = 0, \dots, 4$, constant thereafter. Each curve is attained by one policy under its observation contract.

PROOF. All-fresh attains (24, 24). For (32, 16), cache the 8-unit job; on match complete the other fresh, and on mismatch prepare/cache it. The latter branch certifies the sole write. Theorem 1 forbids $L < 16$. If a policy has $L < 24$, its no-write path contains a matching cached call; invalidating it costs at least $24 + 8 = 32$ by Lemma 2. Together with mandatory work 24 these bounds give the visible frontier.

For erasure, follow any admitted program's no-write run. Cheap would permit an extra failed call, so both completions are guarded. Partition them into matching cached C and fresh/already-stale cached F. If F is nonempty, either both bodies are already protected or a single own-gate write at the remaining matching cached job protects its body too. Both branches publish the same object z_i and retain no compared input or flag. The adversarial identity disappears from the live entry; existing snapshots/records are unchanged. Subsequent observations and choices therefore coincide inductively, preserving protected F bodies. Thus worst $L \geq 24$ is unavoidable.

If $L < 24$, both jobs must instead be matching cached on that path. One fresh write at the 16-unit job forces $W \geq 24 + 16 = 40$ and $L \geq 16$, by Lemma 2. Immediate prepare/cache for both attains (40, 16): at most one comparison can mismatch. These cases exhaust programs and prove the erased frontier.

For the DAG, process Z, A, U, V cheaply until one failure, then prepare/cache. Visible, if Z completed cheaply and A, U both mismatched, finish V fresh *before preparing it*. This trigger certifies three writes and permits $L = 7 = \Omega_2$. If the failure was at Z , duplicate work is at most

2 + 9 = 11; otherwise it is at most 5 + 6 = 11. Thus $W \leq 20$. Erasure permits all three chain duplicates, giving $W \leq 21$; at $B = 3$ at most two can mismatch, giving 20. Earlier values follow from the maximum-job bound.

For any admitted unaware program, one useful write at A forces a cheap failure. In this one-write reference execution, $L = 0$; every remaining completion is matching cached, since another cheap call can be failed in a finite extension. Visible, another write at A protects 5 units, saturating the two-write protection cap. Thus U cannot be fresh; a third write forces its cached mismatch and $W \geq 9 + 5 + 5 + 1 = 20$.

Erased, couple the reference execution with one additional gate write at each of A, U, V , each spent at most once. Every cached return publishes the same z_i and erases the compared input and flag. The same live map, saved outputs, old records and locals are restored; intervening inspections/preparations therefore agree inductively. All three cached calls mismatch, and distinct charges give $W \geq 9 + 5 + 5 + 1 + 1 = 21$. This argument covers arbitrary early preparations/records; it does not invoke visible-tree completeness. The supplement proves the integer family $(M, 1, 1, 2)$ when $r(M - 2) \geq 3$, including the forward coupling [23]. $\square$

A complete erased-observation frontier for general positive-retry DAGs remains open.

7.2 A Native Attempt Is Not Necessarily One Call

If a cached mismatch consumes two units rather than one, set $Q' = Q + \# \text{cached mismatches}$. On completion $Q' = n + z$, where z counts cheap failures plus cached mismatches.

PROPOSITION 8 (A CHANGED CALL UNIT). *Among B -unaware policies satisfying $Q' \leq n + r$ at every finite budget, the least protected-work curve is 0 for $B < r$ and Ω for $B \geq r$. One cheap-then-fresh policy attains the whole curve. With supplied $B > r$, all-fresh also attains the complete frontier (Ω, Ω) .*

PROOF. For the unaware lower bound, invalidate every otherwise-matching comparison, stopping at $z = r$. Before this point all completed bodies are protected. Afterwards, another cheap or cached call is already bad or can be made bad by one extra fresh write, violating universal Q' admissibility. Thus remaining completions are fresh. The hypothetical write is absent from the witness, which spends at most r writes. Admissibility forces termination; knowingly stale failures cannot help. Cheap until r failures then fresh attains the curve.

When $B > r$ is supplied, use the same writer with a cap at $z = r + 1 \leq B$. The cap forbids reaching that count, so every completed body is protected. Mandatory work and all-fresh give the sole frontier point. The archive expands these arguments for retained records and other supplied budgets [23]. $\square$

Maldonado et al.'s irrevocable transition validates an executed prefix, retaining it on success and restarting irrevocably on failure [44, Section III.D]. Specializing the prefix to a whole kernel gives the cached body charges, but grouping the failed transaction and restart into one episode gives Q , while counting both attempts gives Q' . Their mechanism also allows partial prefixes: computing u of a w -unit kernel before protection gives protected work $w - u$ on validity, outside our whole-kernel modes. Our lower bounds do not cover that larger action language.

8 Actual External Writes in Java

Java 17's map API and the pinned implementation supply the per-key synchronization premises below [18–20]. The transfer is conditional; finite executions are not a JDK/JMM proof.

Use Section 2's whole kernels, captured inputs, saved parents and prior store, with one controller and permanent distinct keys. Values are deeply immutable, non-null and identity-equal; replacement may call equals. Key methods are stable, side-effect-free and normally terminate. Kernels make

no recursive/other-key updates. Writers preserve shape/milestones and admit references fresh to all preceding target records/snapshots. Direct locking and retained guards are excluded. Selected operations terminate; waiting is uncharged.

Cheap wraps replace or validation-only compute, exposing success/failure; cached returns its saved output/comparison Boolean; fresh always computes. Observations retain reference equality, payloads, wrapper results, saved parents and selector state. Raw cheap-mismatch values, numeric IDs, clocks and diagnostics are hidden. Erasure requires Section 7.1's constant per-job output and full-history contract; hiding only the flag can leak through result identity.

Correspondence. An injective map η preserves references in the live map, available records, saved results, pending writes and ghost computation history. Only actually available handles enter $\mathcal{R}$; a discarded ghost record cannot be selected. Set $\mathcal{M}(i) = \eta(\text{done}[i])$ from the saved output, and map a record to $(i, \eta(s), \mathcal{M}|_{\text{Pred}(i)}, \eta(z))$. Let Δ queue committed external assignments deferred during normalization. At boundaries,

$$\text{decode}_{\eta}(H.\text{map}) = \text{apply}(\Delta, x), \quad b_H = b_x + |\Delta|.$$

Here b counts committed external publications, and Δ replays assignments rather than callbacks. Retrieval returns a per-key value at a point during its call; replacement checks/stores atomically. compute invokes remapping once, excluding same-key publications through its store. Wrappers save its result even if the map changes before return. These API/source premises cover list/tree bins.

THEOREM 9 (CONDITIONAL RESOURCE TRANSFER FOR ACTUAL WRITES). *Under these premises and each result's observation contract, the body-work guarantees and lower bounds of Sections 3, 4 and 5 hold for actual external writes, with separate or shared bins. Completed-operation traces preserve declared results and Q, W, L , operation kinds and cached-mismatch labels; hence they also preserve Q' of Section 7.2.*

PROOF. *Upper-bound projection.* Group each completed *unprotected* kernel with its captured operands, returned record and full charge as Prepare. Defer writes since capture in Δ , flushing them before the next operation. The record/charge is placed after the source kernel returns, not at an unfinished prefix. For a standalone inspection, flush commits through its read point before that read; queue later commits, preserving the saved return. A protected body stays inside its fresh/cached operation. Other-key commits during that operation may precede its abstract effect: the kernel has no live other-key reads and saved parents are immutable. Same-key writes are excluded until publication.

Post-publication/pre-return writes enter Δ ; completion saves the prepared output for successful cheap calls and the returned output for callbacks, never a map reread. Flush any terminal queue as a zero-foreground-cost write suffix. Resize/tree conversion copies references; a concurrent publication remains a write. This completed-trace normalization preserves declared foreground observations and all charges without increasing writes. It is not an instruction-level simulation or equivalence of writer diagnostics. Explicit upper policies contain bounded finite segments, so selected-operation progress gives completion and transfers their bounds.

Concrete lower witnesses. After the controller fixes its job/mode and prior store, suspend it before API entry. With no foreground guard held, a terminating own-key writer installs a fresh reference and finishes, even with colliding bins. It invalidates every retained target record. Fix these concrete writer results at the restricted histories used in the lower proofs. The target adversaries preserve their unconditional write caps, and Lemma 2's global distinct charges apply to general programs, including early preparation. For a path needing d writes, its witness stops off-path and after d writes on every history. These particular realizable witnesses suffice; arbitrary abstract environments need

not all be implemented in Java. Saved parents preserve readiness, and callback Booleans provide the permitted branches.

The source-to-gate projection preserves operation-kind and cached-mismatch ghost labels, giving $Q' = Q + \# \text{cached mismatches}$. This is separate from program-to-tree domination. The controller may count its visible calls without observing hidden resource labels. $\square$

Earlier kernels perform $8w_i$ iterations; newer wrappers use w_i . Waiting, allocation, traversal and writer work are uncharged. Source control points, transitions and the full relation are archived [23]. Finite tests establish neither API premises nor concurrent-resize correctness.

9 Formal Corroboration and Operational Evaluation

Executions use Apple M2/24 GiB, Python 3.10.9/3.12.14 and OpenJDK 17.0.19; the pinned map source was inspected, not rebuilt. Hashes bind code, inputs and traces. Codex drafted proofs, implementations, input generators, checker/analysis scripts, figures and artifact packaging, and ran experiments. Claude supplied technical critiques and fixed-manuscript reviews. These are author-side AI-assisted checks; no human study or independent certification is claimed.

Inputs fix graph/works/prices; roots add budget/retries. Paths are complete branches; executions are native invocations. Pairs compare policies; coordinates add budgets. Studies overlap.

9.1 RQ1: Proofs, Filters and Their Limits

The retained-record game uses (S, R, b, a) : unfinished jobs, paid valid records, remaining writes and failed-call allowance. It permits preparation at any ready unfinished job. All forward-edge DAGs and weight vectors for $n \leq 4$, $w_i \in \{1, 2, 4\}$, $r = 0, 1, 2$, $B = 0, \dots, n + r$ give 265,629 agreeing call/protection roots from 5,421 inputs. Dominated finite-game moves are omitted; this does not replace the general-program lower proof. A separate fixed study of 647 independent roots and 34,814 interpreted paths agrees with the least unknown-budget work formula.

Table 2. New residual checks. Counts are not independent samples of software workloads. All failures and revisions are retained.

Question	Method, denominator and outcome
Exact safe region	Backward game on 493 roots: 95,514 states/149,472 actions with $k \leq D $, including 936 fresh permissions beyond the earlier rule. Another 62,949 surplus-credit states/154,062 actions are retained. No disagreement in either group.
Python implementation	Two heaps versus direct sorting: 8,355 valid initial states, 58,485 queries, 166,372 actions and 48,000 stream steps agree. Rejected inputs and 9,083 unsafe fresh requests are retained.
Whole work curves	16,267 fixed roots and 1,565,245 independently interpreted paths: new and earlier safe filters have identical worst- W curves in every root. Both retain the all-cached upper bound.
Charged locality	Exact backward thresholds for 16,263 common-fee roots and three added cases. All 379,647 physical common-fee states agree with $T_k(D)$; unrestricted-fee examples violate its necessity.

The 493-root region study uses all forward DAGs and weights $\{1, 3, 7\}$ for $n \leq 3$, plus four declared $n = 4$ weight vectors. Its states need not be reachable from a safe initial policy. The whole-curve study instead fixes all forward DAGs up to four jobs, weights $\{1, 3, 7\}$ and $r \in \{0, 1, 2\}$, plus four diagnostic instances. Both filters share the same proposal rule: minimum-ready cheap; before a mismatch maximum-ready cached; afterwards maximum-ready fresh when permitted, otherwise

minimum-ready cached. Thus strict action permissiveness is observed, but no new worst-work gain is established by these 16,267 comparisons. A previous prediction that the new rule would worsen one curve was also falsified and retained. Separate DAG exploration found a local choice with crossing suffix-work curves, yet other histories made the complete initial curves identical. A general positive-retry DAG optimum remains open.

The charged game independently computes the largest viable incurred cost $V(D, k)$, with terminal value G_k , using

$$V(D, k) = \max_{i \text{ ready}} \max\{V(D + i, k) - p_i, \min\{V(D + i, k), V(D + i, k + 1) - q_i\}\}.$$

It does not substitute the proposed invariant. Physical states satisfy $k \leq |D|$; surplus-credit states are kept separately and sometimes refute an extension beyond that domain. The original reachable fee pair uses a future $q = 1000$ with fresh costs 990 or 999; Proposition 6's smaller integer pair is a later analytic simplification, not a replacement of those fixed experimental inputs. Tests support implementation fidelity, not universal proofs or importance.

Java implementation of the residual interface. The Java17 component receives only completed weights, k, ℓ , the current weight and the public outcome. Two linked complete binary heaps support the stated abstract operation bounds; long overflow is rejected, with allocation/GC time unbounded. All 498 operation sequences and two heap regression cases match a separately written sorting reference. A per-key compute integration executes 996 complete paths on 48 fixed roots; all 31,892 events and saved outputs match the reference. The chain from Section 5 exhibits the strict fresh permission. Each of 118 postpublication paths and 118 double-write paths, included in those 996, checks saved-output persistence or distinguishes actual writes from observed mismatches. All 62 worst- W coordinates equal the same cheap phase followed by all-cached completion. All-fresh is also executed, but violates the common zero-write protection contract. These are authored whole-kernel fixtures, not an actual application client or a proof of all Java interleavings. Kernel work excludes filter maintenance, allocation, parent capture, map internals, writers and waiting; no latency benefit is inferred.

Earlier Java wrappers cover colliding/separate/tree bins and canonical/allocating outputs. All 6,112 supplied and 10,192 unaware pairs agree. Twelve corruptions reject; four of 52 flag-ignoring controls violate protection. The unaware independent selector sees no budget, cached flag or diagnostics. The DAG observation family has 21 roots, seven controls and 9,373 paths; both faulty policies reject. Changed-call checks include 13,176 supplied roots and 222 unaware curves. These overlapping studies and the older supplied-frontier, tree-domination and charged-complexity checks remain in the archive.

9.2 RQ2: A Native Traversal and Its Guard Boundary

Linux v6.12 `dentry_path_raw` first walks mutable `dentry` parents/names under RCU, validates `rename_lock`'s sequence, and on failure rebuilds under its reader-exclusive lock [7]. Pathname lock contention motivated the upstream optimistic fallback [9]. The related `is_subdir` comment explicitly connects one lockless try with fallback for progress [8]. In this pathname loop, the caller sets the marker to 1 after failure; the primitive does not automatically count retries. Our KUnit-only variant prepares a second traversal with a *new even* sequence, then acquires the lock and validates that saved sequence: reuse on match, reset/rebuild on mismatch. Every traversed component must be initialized, with no concurrent initialization; a valid reference alone is insufficient. Initial `dentry/name` publication must precede parallel reading; subsequent relevant changes must participate in the global sequence, without wrap. This covers the initialized `d_exchange` fixtures, not every

filesystem: `d_mark_tmpfile` changes names without that sequence during initialization [8]. Mount traversal and `getcwd` are excluded.

Assume a private valid buffer with $1 \leq \text{capacity} \leq \text{INT_MAX}$, nonnegative observed lengths fitting `int`, valid dentry/RCU lifetimes, and successful stable-name copies. Let $C = \text{capacity} - 1$ after NUL. In the original factored loop, attempted names v , requested copy bytes c , fault-fill bytes f and successful slashes s satisfy $v \leq s + 1$, $f \leq c$ and $c + s \leq C$. Thus body events $T = 1 + v + c + f + s \leq 2C + 2$. For first/second validation failures X, Y , fresh even capture gives $X + XY \leq \min(B, 2)$ for actual global writer sections, including unrelated renames. Count one Q unit per lockless validation or guarded validation/completion block, not per native API call. The candidate has $Q \leq 2$, $T_{\text{total}} \leq (1 + \min(B, 2))(2C + 2)$ and $L_T = 0$ for $B \leq 1$. This is a single-job upper refinement: private-buffer return does not instantiate the full publication/observation contract. Guard/RCU instructions, waiting and postambles are separate; the capacity-4096 ceiling 8192 is loose on short paths.

ARM64 Linux runs in two-CPU QEMU with lockdep and atomic-sleep checking. A pinned native `lock_rename/d_exchange` writer uses endpoint gates outside RCU and a nonsleeping mid-parent gate. The fixed matrix has 1,920 rows (948 paths/972 expected overflows) and four deliberate faulty variants, all detected. A fixture oracle checks strings, offsets, canaries and post-return persistence against quiescent upstream. Outputs/body records agree without lock/RCU diagnostics. Even nonwrapping sequence brackets confirm designated writer counts and all prior output/body rows. Returning unvalidated overflow, failing to restore the buffer or ignoring final validation breaks output. Reusing the first sequence preserves output but violates the protection bound at $B = 1$. No copy-fault/allocation failure is injected; native-time performance is unmeasured.

Table 3. Source event counts at the scripted gates. Initial path is `/driver/cache/entry/leaf`; first exchange moves it to `/alternate`, second restores it. G counts reader-exclusive acquisitions.

Policy	One write				Two writes			
	T	L_T	Q	G	T	L_T	Q	G
Upstream	41	12	2	1	58	29	2	1
Cached	41	0	2	1	70	29	2	1
Two cheap, then locked	41	0	2	0	70	29	3	1

Count a body attempt whenever reconstruction starts from the original dentry/buffer, including an overflow exit. Upstream has $1 + X \leq 2$ attempts; cached has $1 + X + XY \leq 3$. The latter preserves bounded completion under terminating primitives but relaxes upstream's two-attempt bound. It is not a proof of faster completion. All 384 cached/two-cheap pairs agree on output and body vectors; 138 first-invalid/second-valid pairs avoid the guard, while 144 two-failure pairs need another logical block. The cap comparison depends on its unit. One public `dentry_path_raw` invocation remains one under every policy. Counting six named primitives—RCU enter/exit, sequence begin/retry, and exclusive lock/unlock—gives $4R + 2G$, where R counts outside traversals. The cached/two-cheap counts are 4/4, 10/8, 10/10 for zero, one and two observed failures: both universal maxima are 10, despite logical caps 2/3. These are source call counts, excluding spinning and underlying instructions. No caller-required $Q = 2$ SLA was found. A stronger native name-snapshot comparator copies inline names or retains external-name references under protection, then materializes outside. Its allocation, dentry locks, capture steps and refcounts are separate from L_T ; five 64-slot overflows retain the discarded capture work before protected fallback. Its body-only zero is not zero total protected work or a lower bound for larger workspaces. All adverse results and the first 1,536-row matrix remain; the populations overlap.

9.3 RQ3: Preservation and Cost Limits in Roslyn

Roslyn’s `Workspace.cs` at `6c4a46a3` transforms an immutable solution outside a semaphore, validates identity inside and retries [10, 11]. Our `SourceText`-only patch switches after r shared failures to fresh (two-mode) or cached completion (three-mode). A stronger rebase reuses guarded document contents but still requires a protected merge. Lazy work, allocation, version generation and cache-dependent merges prevent fixed-work transfer. No upstream numerical cap or SLA was found.

All 236 authored cases agree; 12 corruptions reject. Full event/publication/snapshot records are archived. In 96 compiler processes, 2,556 project projections match declared expectations for references, parse options and linked text under cold/warm caches; eight corruptions reject. The original same-text deadlock and four missing-document timeouts remain archived; corrected patches pass 14 error cases and reentry controls. These selected checks use Roslyn on both sides. Custom loaders, changed-text reentry and general concurrent equivalence remain outside scope.

Arrival tests compare shared/per-update caps, two/three modes and both baselines. Eight balanced blocks serve 32 updates and 64 FIFO writers each at 0, 10, 100, 1000 μ s. All 2,560 measured and 512 warm-up units pass source/count checks; 12 corruptions reject. At $r = 2$, local/shared caps allow 96/34 calls, precluding equal-cap comparison.

Shared three/two foreground-time ratios at 0/10 μ s are 1.72 [1.40,2.11] and 1.58 [1.17,2.16], favoring two modes. Slower-arrival and all writer-response intervals include 1. Geometric means use eight paired block ratios and descriptive 95% intervals from 10,000 fixed block-bootstrap samples without multiplicity correction. Baseline ratios 0.83–1.08 all include 1; instrumentation overhead is unidentified. Authored arrivals, eight blocks and one host limit inference.

10 Related Work and Research Lineage

The contract and existing mechanisms. Maldonado et al. [44, Section III.D] retain a valid transaction prefix on becoming irrevocable, or restart irrevocably after invalidation. Our cached operation uses that established protection/reuse idea; the new object is its resource guarantee under a whole-kernel completion-call contract. Their partial-prefix migration and native transaction attempts do not obey that contract automatically. Section 7.2 makes the accounting difference explicit.

Fallback and contention. Protected reexecution [6, Section 3.3], bounded lock-elision/RPC retries [26, 27, 31, 32], irrevocable prefix preservation [33], contention management [4, 24, 25] and switching/tuning [5, 41, 42] are established. HTM aborts need not consume writes [28], and RPC can fail. Their progress/time objectives differ from persistent-job W/L caps.

Synthesis. Černý et al. [15] synthesize retries, guards and caches for weighted limit-average safety (PSPACE by erratum); later work optimizes protection/profile costs [16] or preemption-safe locks [17]. We inherit global-budget/information-state minimax DP [12, 21, 29], backward induction and Pareto composition; our lower bounds supply an exact independent-job work curve, and our residual theorem replaces game enumeration for one explicit all-budget protection contract.

Scheduling with testing. At $r = 0$, fresh is untested execution with $u_i = w_i$; cached tests for $t_i = w_i$, then executes for $p_i \in \{0, w_i\}$. Thus W counts testing plus execution; L counts execution. For any deterministic tree, hidden bad-job labels represent its mismatch set and own-gate writes realize them. Dynamic writing alone does not distinguish this canonical game.

Dürr et al. [35] introduce adversarial testing; Buld and Schulz [38] and Albers and Eckl [39] allow unequal test times/upper bounds under competitive time objectives. Damerius et al. [36] separate testing and execution with a *policy-side* budget; ours bounds adverse outcomes and permits preparation at $B = 0$. Dufossé et al. [37] study minimax testing, revealing durations even for untested

jobs; polynomial adaptive optimality assumes their two-phase conjecture. Albers and Janke [40] bound failing jobs for irrevocable parallel-machine assignment.

Our absolute W/L caps include retry slack. For the 8/16 adaptive and cache-both policies, $(\sup T, \sup L) = (24, 16)$, where T counts preparation, yet $\sup(T + L) = 32$ and 40. The distinction is the absolute joint contract and its simultaneous all-budget quantifier. Adaptive testing and binary policy trees are inherited; dynamically placing a write is not by itself a novelty claim.

Budgeted sums and safety filters. Bertsimas and Sim’s cardinality-budget protection function [14, Section 3] supplies the top-weight sum. Our sorted-union identity is elementary, not a new robust-optimization method. Bloem et al. [45, Sections 3–5] enforce safety through a winning region; allowing only winning successors is inherited. Feng et al. [46, Sections 4.4–4.5] enlarge control envelopes through runtime parameter inference, while Cano et al. [47, Appendix A] construct maximally permissive shields with history. Our game can be encoded in these general control frameworks. The specific result is that its exact region discards all future job data, with a proved fee boundary where that interface loses exactness. Our observed mismatch count is a certain write lower bound, not a statistical estimate or a theorem about minimum history in general shields.

Precedence and reuse. The supplementary charged frontiers use established precedence [34] and scheduling-hardness constructions [1]; they do not prove general DAG work optimality for the present filter. Universal restart [43] concerns expected Las Vegas time. Build and incremental systems [2, 3, 30] may reuse partial work outside our whole-kernel interface.

11 Limitations and Threats to Validity

Supplied B is a premise. Native costs are uncalibrated; instrumentation, caches and runtime compilation limit timings. The archive retains six scaling timeouts, retained-guard/simple-rule counterexamples, reversed disjoint-bin timings, scalar gains in only 2/96 cells and successful cost fits in 2/8. All-budget compilation can cost more than requested-value computation. Against reader-repaired Valkey digests, cooperating writer-maintained materialization wins time/transfer in all 16 cells, with mixed maximum command times. Such larger interfaces delimit our claims. Finite/AI-assisted checks are neither mechanical proofs nor independent reproduction; human benefit is unmeasured.

12 Conclusion

One policy can attain the exact least total-work curve under simultaneous optimal protection for independent jobs. For any DAG, an exact residual filter separates safe mode selection from future job data; fee accounting determines when that separation survives. These results characterize a resource interface built from established mechanisms. Conditional Java transfer, changed native call units and adverse Linux/Roslyn comparisons delimit its application.

Data Availability

Code, fixed inputs, proofs, all outcomes and corrections, Java sources/traces, replay commands and licenses are available in the anonymous artifact [48]. The cited immutable commit includes 57 standard stages for reproducing fixed evidence, with separate native rebuild instructions. The studies and replays are author checks, not independent certification or additional native timing samples.

References

- [1] Michael R. Garey and David S. Johnson. 1976. Scheduling Tasks with Nonuniform Deadlines on Two Processors. *Journal of the ACM* 23, 3, 461–467. Theorem 2 (pp. 465–466) concerns one processor. <https://doi.org/10.1145/321958.321967>.
- [2] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems à la Carte. *PACMPL* 2, ICFP, Article 79, 1–29. <https://doi.org/10.1145/3236774>. Author version: <https://simon.peytonjones.org/assets/pdfs/build-systems-original.pdf>.
- [3] Ruy Ley-Wild, Umut A. Acar, and Matthew Fluet. 2009. A Cost Semantics for Self-Adjusting Computation. In *POPL*, 186–199. <https://doi.org/10.1145/1480881.1480907>. Author version: <https://ttic.uchicago.edu/~pl/sa-sml/popl09.pdf>.
- [4] Rachid Guerraoui, Maurice Herlihy, and Bastian Pochon. 2005. Toward a Theory of Transactional Contention Managers. In *PODC*. Author version. <https://lpdwww.epfl.ch/upload/documents/publications/neg--509904679f181-pochon.pdf>.
- [5] Nuno Diegues and Paolo Romano. 2014. Self-Tuning Intel Transactional Synchronization Extensions. In *ICAC*, 209–219. <https://www.usenix.org/conference/icac14/technical-sessions/presentation/diegues>. Errata.
- [6] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. <https://doi.org/10.1145/319566.319567>.
- [7] Linux contributors. 2024. Linux v6.12, fs/d_path.c and fs/hostfs/hostfs_kern.c, commit adc218676eef25575469234709c2d87185ca223a. Accessed September 11, 2026. https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/d_path.c.
- [8] Linux contributors. 2024. Linux v6.12, fs/dcache.c: name snapshots, d_exchange, and tmpfile initialization. Accessed September 11, 2026. <https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/dcache.c>.
- [9] Waiman Long. 2013. dcache: Translating dentry into pathname without taking rename_lock. Linux commit 232d2d60aa5469bb097f55728f65146bd49c1d25. <https://github.com/torvalds/linux/commit/232d2d60aa5469bb097f55728f65146bd49c1d25>.
- [10] .NET Foundation. 2025. *Roslyn source*, commit 6c4a46a31302167b425d5e0a31ea83c9a9aa1d09. Workspace.cs, Solution.cs, SolutionCompilationState.cs and SolutionState.cs. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/tree/6c4a46a31302167b425d5e0a31ea83c9a9aa1d09>.
- [11] .NET Foundation contributors. 2020. Add SetCurrentSolution with solution transformation, pull request 42228. Source review on replayability and event ordering. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/pull/42228>.
- [12] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [13] Avi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [14] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [15] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [16] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*. <https://arxiv.org/abs/1511.07163>.
- [17] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2017. From non-preemptive to preemptive scheduling using synchronization synthesis. *Formal Methods in System Design* 50, 97–139. <https://doi.org/10.1007/s10703-016-0256-5>.
- [18] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [19] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [20] OpenJDK. 2026. ConcurrentHashMap.java, jdk17u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.
- [21] Krzysztof Postek, Ward Romeijnnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. Online appendix; author code.
- [22] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e0110. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbbf422390ad8d29785ad0023f>.

- [23] Anonymous Authors. 2026. *Retry Synthesis with Bounded Calls and Protected Work: Research Artifact*. Total-work, comparison-free and unit-guard precedence, and call-accounting supplement, commit b719eb7. <https://github.com/research-artifact-archive/research-artifact/tree/b719eb7a6514d7f3a47f2a3af21e43b4fffd4699f>.
- [24] Hagit Attiya, Leah Epstein, Hadas Shachnai, and Tami Tamir. 2006. Transactional Contention Management as a Non-Clairvoyant Scheduling Problem. In *PODC*. <https://cs.idc.ac.il/~tami/Papers/podc06.pdf>.
- [25] Dan Alistarh, Syed Kamran Haider, Raphael Kübler, and Giorgi Nadiradze. 2018. The Transactional Conflict Problem. *arXiv preprint arXiv:1804.00947v1*. <https://arxiv.org/abs/1804.00947>.
- [26] Free Software Foundation. 2025. GNU C Library 2.42: x86 lock elision, revision d2097651. `elision-lock.c` and `elision-conf.c`. <https://github.com/bminor/glibc/tree/d2097651cc57834dbfcaa102ddfcae0d86cfb66/sysdeps/unix/sysv/linux/x86>. Official tunables: https://www.gnu.org/software/libc/manual/html_node/Elision-Tunables.html.
- [27] OpenJDK. 2025. HotSpot 21.0.8+9 RTM monitor code, revision 54f20959. `c2_MacroAssembler_x86.cpp` and `globals_x86.hpp`. <https://github.com/openjdk/jdk21u/tree/54f2095960f01d957f2335cafa0defb956e13a2c/src/hotspot/cpu/x86>.
- [28] GNU Compiler Collection. n.d. x86 Transactional Memory Intrinsics. Accessed September 10, 2026. <https://gcc.gnu.org/onlinedocs/gcc/x86-transactional-memory-intrinsics.html>.
- [29] Aditya Dave, Nishanth Venkatesh, and Andreas A. Malikopoulos. 2024. Approximate Information States for Worst-Case Control and Learning in Uncertain Systems. *arXiv preprint arXiv:2301.05089v2*. <https://arxiv.org/abs/2301.05089>.
- [30] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. 2014. Adaption: Composable, Demand-Driven Incremental Computation. In *PLDI*. <https://doi.org/10.1145/2594291.2594324>. Author version: <https://www.cs.tufts.edu/~jfoster/papers/pldi14.pdf>.
- [31] Mike Ulrich. 2016. Addressing Cascading Failures. In *Site Reliability Engineering*, Chapter 22, Retries. <https://sre.google/sre-book/addressing-cascading-failures/>.
- [32] Envoy contributors. n.d. CircuitBreakers.Thresholds.RetryBudget. Accessed September 10, 2026. https://www.envoyproxy.io/docs/envoy/latest/api-v3/config/cluster/v3/circuit_breaker.proto.html.
- [33] Adam Welc, Bratin Saha, and Ali-Reza Adl-Tabatabai. 2008. Irrevocable Transactions and their Applications. In *SPAA*, 285–296. <https://doi.org/10.1145/1378533.1378584>.
- [34] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. <https://doi.org/10.1287/mnsc.19.5.544>. Author technical report ERL-M327 (1971): <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1971/Archive/ERL-m-327.pdf>.
- [35] Christoph Dürr, Thomas Erlebach, Nicole Megow, and Julie Meißner. 2020. An Adversarial Model for Scheduling with Testing. Author preprint arXiv:1709.02592v3. <https://arxiv.org/abs/1709.02592v3>.
- [36] Christoph Damerius, Peter Kling, Minming Li, Chenyang Xu, and Ruilong Zhang. 2023. Scheduling with a Limited Testing Budget: Tight Results for the Offline and Oblivious Settings. Author preprint arXiv:2306.15597v1. <https://arxiv.org/abs/2306.15597v1>.
- [37] Fanny Dufossé, Christoph Dürr, Noël Nadal, Denis Trystram, and Óscar C. Vázquez. 2021. Scheduling with a Processing Time Oracle. Author preprint arXiv:2005.03394v3. <https://arxiv.org/abs/2005.03394v3>.
- [38] Felix Buld and Andreas S. Schulz. 2026. Scheduling with Testing: Competitive Algorithms for Minimizing the Total Weighted Completion Time in the Adversarial Model. Author preprint arXiv:2606.25166v1. <https://arxiv.org/abs/2606.25166v1>.
- [39] Susanne Albers and Alexander Eckl. 2021. Explorable Uncertainty in Scheduling with Non-Uniform Testing Times. Author preprint arXiv:2009.13316v2; first version 2020. <https://arxiv.org/abs/2009.13316v2>.
- [40] Susanne Albers and Maximilian Janke. 2021. Online Makespan Minimization with Budgeted Uncertainty. In *WADS*. https://doi.org/10.1007/978-3-030-83508-8_4. Author version: <https://maximilian-janke.de/data/BudgetedUncertainty.pdf>.
- [41] Beng-Hong Lim and Anant Agarwal. 1994. Reactive Synchronization Algorithms for Multiprocessors. In *ASPLOS VI*, 25–35. Author version: <https://groups.csail.mit.edu/cag/pub/papers/pdf/reactive.pdf>.
- [42] Takayuki Usui, Reimer Behrendts, Jacob Evans, and Yannis Smaragdakis. 2010. Adaptive Locks: Combining Transactions and Locks for Efficient Concurrency. *Journal of Parallel and Distributed Computing*. <https://doi.org/10.1016/j.jpdc.2010.02.006>. Author version: <https://people.cs.umass.edu/~yannis/adaptive-jpdc.pdf>.
- [43] Michael Luby, Alistair Sinclair, and David Zuckerman. 1993. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters* 47, 4, 173–180. Author version: <https://www.cs.utexas.edu/~diz/pubs/speedup.pdf>.
- [44] Walther Maldonado, Patrick Marlier, Pascal Felber, Julia Lawall, Gilles Muller, and Etienne Rivière. 2011. Deadline-Aware Scheduling for Software Transactional Memory. In *DSN*. Author version, Section III.D. <https://who.paris.inria.fr/Gilles.Muller/papers/dsn2011.pdf>.
- [45] Roderick Bloem, Bettina Könighofer, Robert Könighofer, and Chao Wang. 2015. Shield Synthesis: Runtime Enforcement for Reactive Systems. Extended author version arXiv:1501.02573v2. <https://arxiv.org/abs/1501.02573v2>.

- [46] Yao Feng, Jun Zhu, André Platzer, and Jonathan Laurent. 2025. Adaptive Shielding via Parametric Safety Proofs. *PACMPL* 9, OOPSLA1. <https://doi.org/10.1145/3720450>. Author version: <https://arxiv.org/abs/2502.18879>.
- [47] Filip Cano, Thomas A. Henzinger, and Konstantin Kueffner. 2026. Shielding for Higher-Order Safety. Author version arXiv:2608.03662v1. <https://arxiv.org/abs/2608.03662v1>.
- [48] Anonymous Authors. 2026. *Resource Contracts for Completing Optimistic Computations: Research Artifact*. Residual-safety and Java implementation supplement, commit 0716120. <https://github.com/research-artifact-archive/research-artifact/tree/0716120a06423e871305bf51825f903ba3ccbde0>.